

실전기계학습 기말 미니 프로젝트

- Convolutional Neural Network(CNN) 구현 -

2026.05.15

조교: 김민성

ENC LAB



Kyung Hee University

목차

- Super Resolution 소개
- 배열을 이용한 CNN 구현 예제
- 기말 프로젝트: 객체지향 CNN 구현
 - Tensor 객체 구현
 - Layer 객체 구현
 - Model 객체 구현
- 제출 방법 및 기간

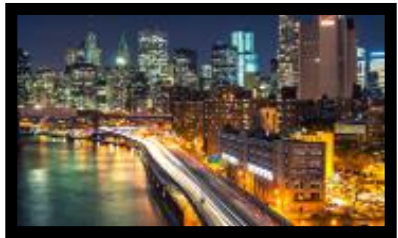


Super-Resolution 소개

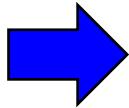
3

- Super-Resolution이란?
 - 작은 크기의 영상을 선명한 고해상도 영상으로 복원하는 기술
 - UHD TV, 차량 블랙박스 번호판 영상 복원, CSI의 CCTV 분석 등에 사용 가능

Just Upsampling



Full-HD content



4K UHD TV with the conventional interpolation

Super Resolution



4K UHD TV with Super Resolution



Super-Resolution 소개

Label-Use2019090111 소개

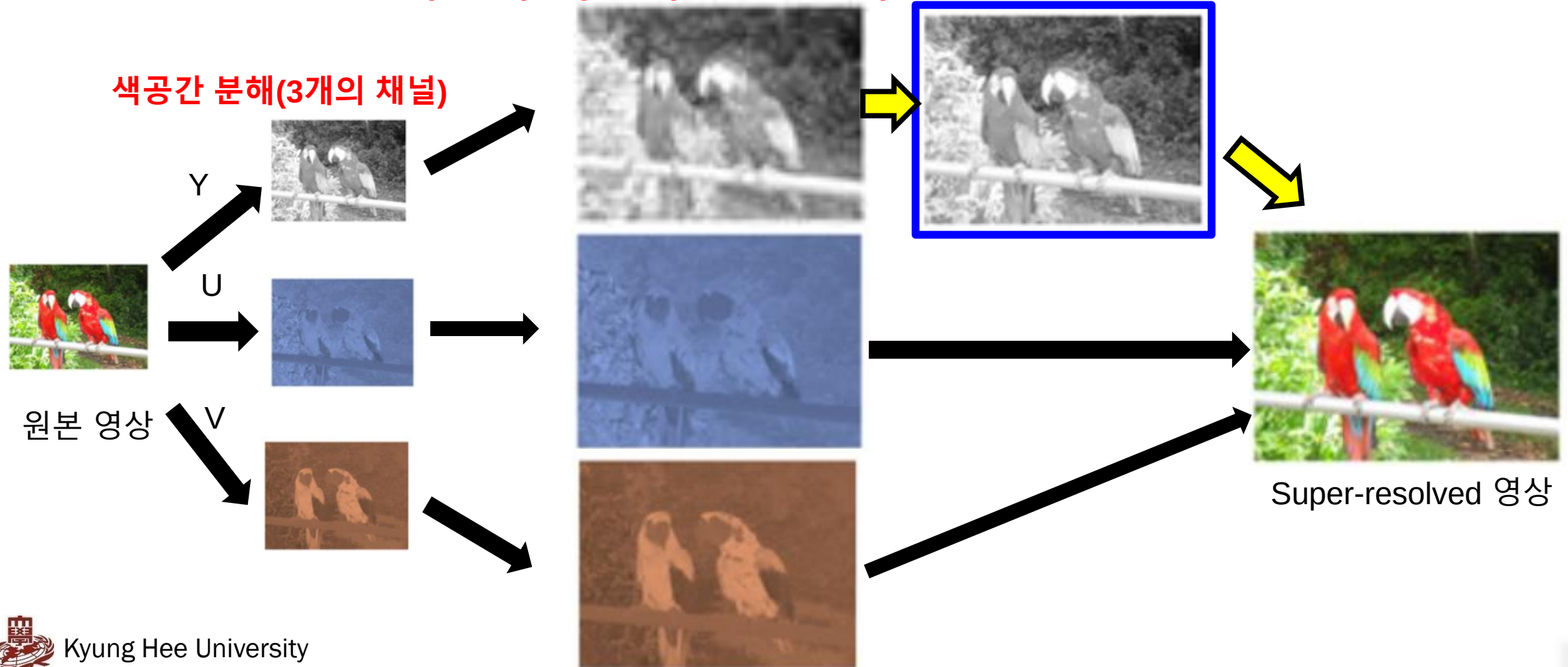
- Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network (CVPR 2017)
 - CNN을 이용한 Super-Resolution(SR)은 괄목할 만한 성능을 보임



Super-Resolution 소개

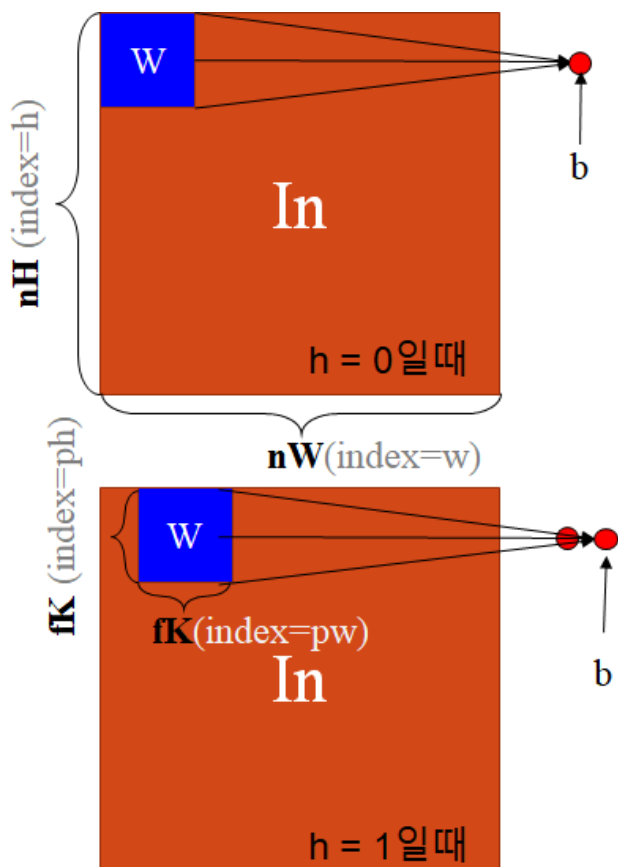
- Super-Resolution은 Y (밝기) 채널에 대해서만 수행(시각적 특성)

Up-sampling 분해(단순 면적 증가) Super-Resolution 수행(우리가 할 부분)



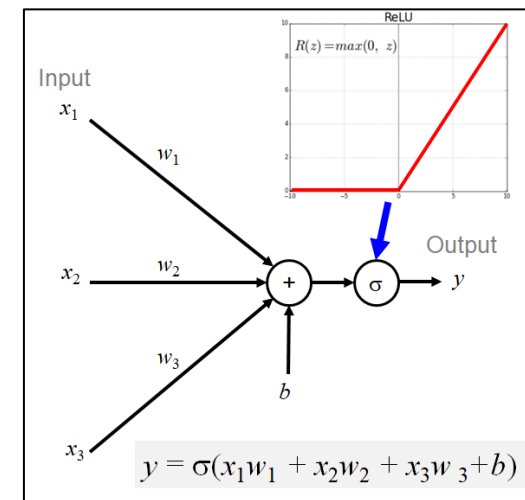
배열을 이용한 CNN 구현 예제

- 2차원 배열 입력에 대한 Convolution Layer 예제
 - 가중치 곱들의 합 ($y = W * X$)을 입력의 모든 부분에 대해 수행



```
double In[nH][nW] ; // nH x nW 크기 입력 이미지
double Out[nH][nW] ; // nH x nW 크기 출력 이미지
double W[fK][fK]; // fK x fK 크기 필터
double b; //bias
```

```
// Convolution
int offset = (fK - 1) / 2;
for (int h = offset; h < nH - offset; h++) {
    for (int w = offset; w < nW - offset; w++) {
        for (int ph = 0; ph < fK; ph++) {
            for (int pw = 0; pw < fK; pw++) {
                Out[h][w] += In[ph+h-offset][pw+w-offset]*W[ph][pw];
            }
        }
        Out[h][w] += b;
    }
}
```

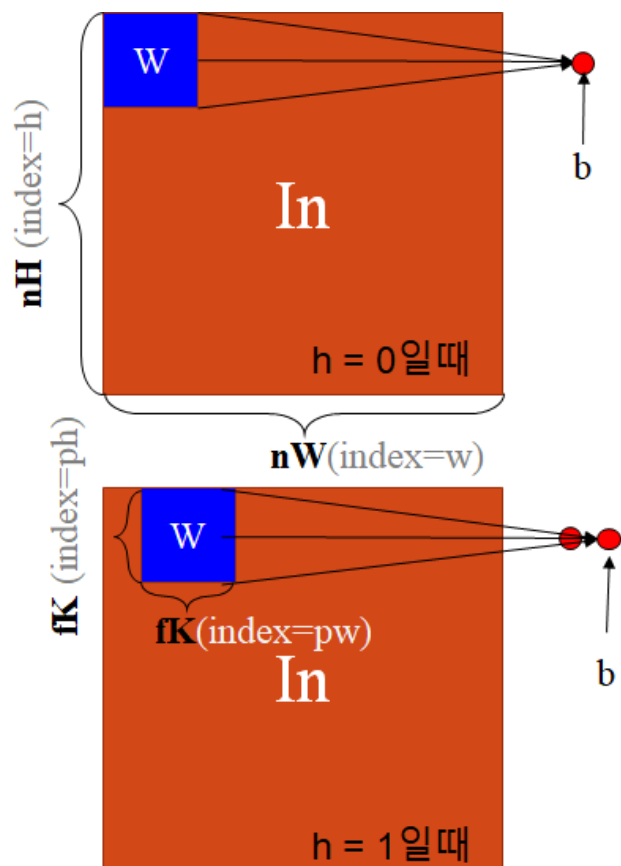


$$y = \sigma(x_1w_1 + x_2w_2 + x_3w_3 + b)$$

배열을 이용한 CNN 구현 예제

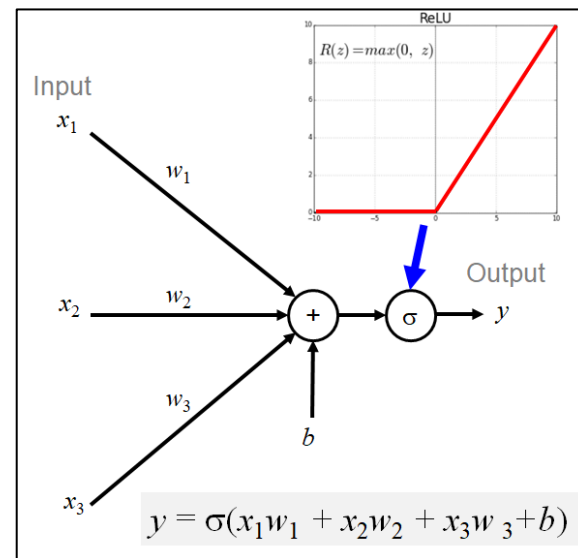
● 2차원 배열 입력에 대한 ReLU Layer 예제

- 가중치 곱들의 합 ($y = W * X$)을 입력의 모든 부분에 대해 수행



```
double In[nH][nW] ; // nH x nW 크기 입력 이미지
double Out[nH][nW] ; // nH x nW 크기 출력 이미지
double W[fK][fK]; // fK x fK 크기 필터
double b; //bias
```

```
// ReLU (출력이 음수면 0으로 보냄)
int offset = (fH - 1) / 2;
for (int h = offset; h < nH - offset; h++) {
    for (int w = offset; w < nW - offset; w++) {
        Out[h][w] = In[h][w]
        if ( Out[h][w] < 0)
            Out[h][w] = 0;
    }
}
```

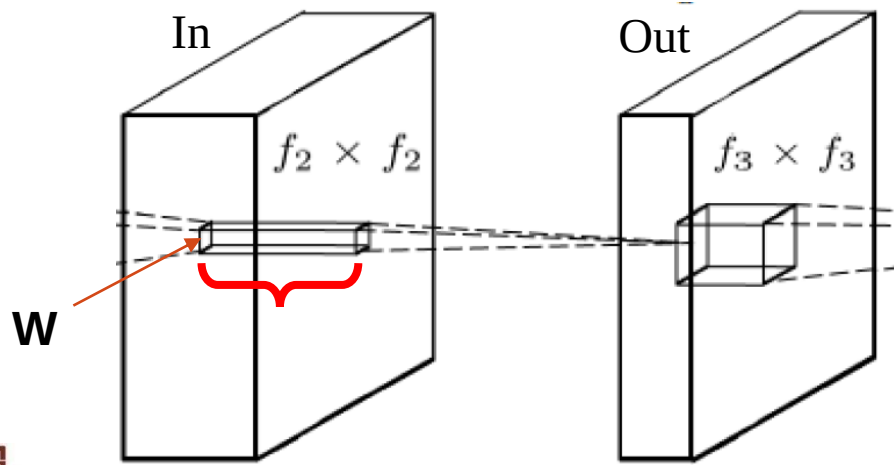


$$y = \sigma(x_1 w_1 + x_2 w_2 + x_3 w_3 + b)$$

배열을 이용한 CNN 구현 예제

- 실제 CNN의 입/출력은 3차원 Tensor, Convolution filter는 4차원 Tensor임
 - 1차원 데이터: vector (예: `double vector[10]`)
 - 2차원 데이터: matrix (예: `double matrix[10][10]`)
 - 3차원 이상 데이터: tensor (예: `double tensor[10][10][10]`)

```
double In[nH][nW][nC_in] ; // nH  nW  nC_in 크기 입력 이미지
double Out[nH][nW][nC_out]; // nH  nW  nC_out 크기 출력 이미지
double W[fK][fK][nC_in][nC_out]; // fK  fK  nC_in  bC_out (fK  fK  nC_in 크기의 필터가 총 nC_outN 개 있음)
double b[nC_out]; //bias는 각 filter마다 1개 (filter의 개수만큼 존재)
```



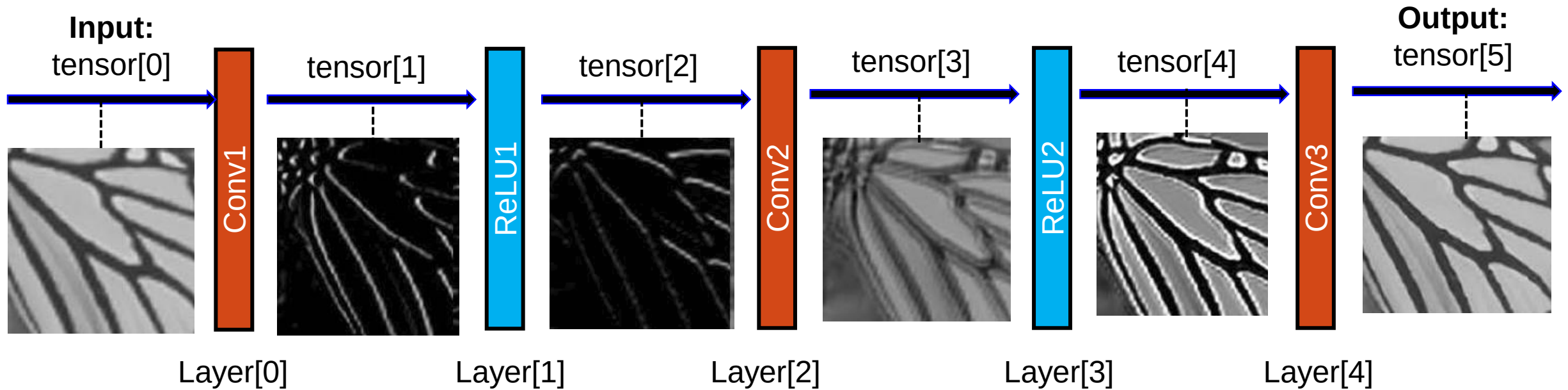
CNN은 3차원 convolution을 수행

1. 입력 영상 (In) 채널의 개수 (`nC_in`)와 filter 채널크기는 같아야 함
2. 출력 영상 (Out) 채널의 개수(`nC_out`)는 filter의 개수와 같아야 함
3. bias (`b`)는 각 filter별로 1개의 값을 가짐

기말프로젝트: 객체지향 CNN 구현

- Tensor 객체

- CNN은 크게 동작을 수행하는 Layer와, 정보를 전달/저장하는 Tensor로 구성
- CNN의 입/출력 및 내부 부산물(중간 출력층 결과물)는 Tensor 객체로 표현
- 비유: Tensor는 물, Layer는 정화조



기말프로젝트: 객체지향 CNN 구현

- Tensor 객체: _1_main_tensor.cpp 가 수행되도록 CTensor.h 를 구현할 것
 - imagelib.h 에 있는 1/2/3/4 차원 동적배열 할당 및 해제 함수를 사용할 것(주석을 읽어볼 것)

```
class Tensor3D {
private:
    double*** tensor;
    int nH; // height
    int nW; // width
    int nC; // channel
public:
    Tensor3D(int _nH, int _nW, int _nC) { ... }
    ~Tensor3D() { ... }
    void set_elem(int _h, int _w, int _c, double _val) { tensor[_h][_w][_c] = _val; }
    double get_elem(int _h, int _w, int _c) const { ... }

    void get_info(int& _nH, int& _nW, int& _nC) const { ... }

    void set_tensor(double*** _tensor) { tensor = _tensor; }
    double*** get_tensor() const { return tensor; }

    void print() const { ... }
};
```

CTensor.h

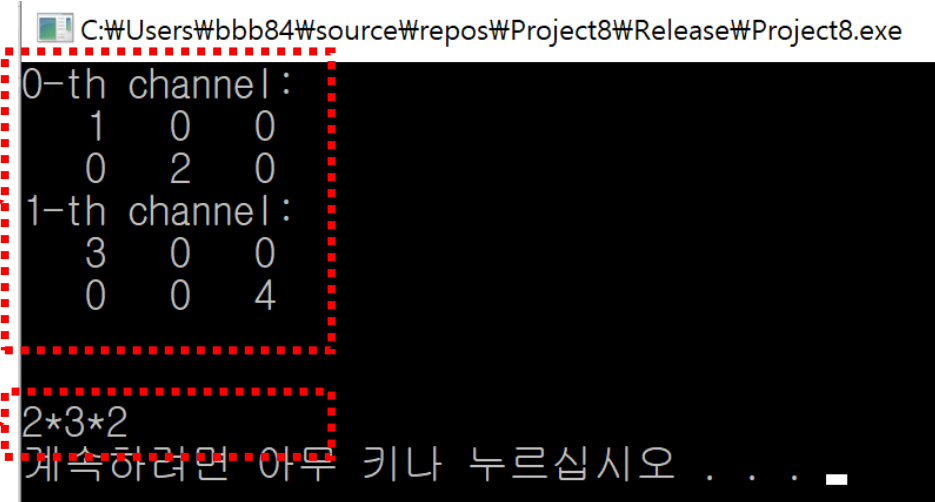


기말프로젝트: 객체지향 CNN 구현

- Tensor 객체: `_1_main_tensor.cpp` 가 수행되도록 `CTensor.h` 를 구현할 것
 - 각 함수별로 실행결과와 일치하게 나오도록 작성할 것

```
int main() {  
  
    int nH = 2; // 행의 갯수  
    int nW = 3; // 열의 갯수  
    int nC = 2; // 채널의 갯수  
    Tensor3D tensor(nH, nW, nC); // nH x nW x nC 크기의 3차원 행렬 생성  
  
    //  
    tensor.set_elem(0, 0, 0, 1);  
    tensor.set_elem(1, 1, 0, 2);  
    tensor.set_elem(0, 0, 1, 3);  
    tensor.set_elem(1, 2, 1, 4);  
  
    print_all_elements(tensor);  
    tensor.print();  
  
    system("PAUSE");  
  
    return 0;  
}
```

`_1_main_tensor.cpp`



```
C:\Users\Wbbb84\source\repos\Project8\Release\Project8.exe  
0-th channel:  
 1  0  0  
 0  2  0  
1-th channel:  
 3  0  0  
 0  0  4  
2*3*2  
계속하려면 아무 키나 누르십시오 . . .
```

실행 결과

기말프로젝트: 객체지향 CNN 구현

- Layer 객체: _2_main_layer.cpp를 구현
 - 이번 단계에서는 Layer_Conv Class의 init() method에서 MEAN_INIT, LOAD_INIT 중 MEAN_INIT만 구현해도 됨
 - MEAN_INIT: Convolution 연산이 입력값에 대한 평균값을 출력하도록 weight를 초기화(이때 bias는 0으로)
 - 예를 들어, 입력이 5x5x3 크기의 matrix일 경우 각 가중치의 값은 $1/(5 \times 5 \times 3)$ 이 됨

```
class Layer {
protected:
    int fK; // kernel size in K*K kernel
    int fC_in; // number of channels
    int fC_out; // number of filters
    string name;
public:
    Layer(string _name, int _fK, int _fC_in, int _fC_out) : name(_name), fK(_fK), fC_in(_fC_in), fC_out(_fC_out) {}
    virtual ~Layer() {} // 가상소멸자 (참고: https://wonjayk.tistory.com/243)
    virtual Tensor3D* forward(const Tensor3D* input) = 0;
    // virtual bool backward() = 0;
    virtual void print() const = 0;
    virtual void get_info(string& _name, int& _fK, int& _fC_in, int& _fC_out) const = 0;
};
```

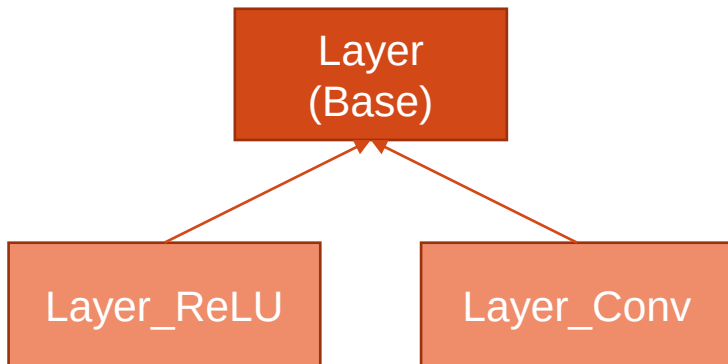
Layer Class

```
class Layer_ReLU : public Layer {
public:
    Layer_ReLU(string _name, int _fK, int _fC_in, int _fC_out) { ... }
    ~Layer_ReLU() {}
    Tensor3D* forward(const Tensor3D* input) override { ... }
    void get_info(string& _name, int& _fK, int& _fC_in, int& _fC_out) const override { ... }
    void print() const override { ... }
};
```

Layer_ReLU Class

```
class Layer_Conv : public Layer {
private:
    string filename_weight;
    string filename_bias;
    double*** weight_tensor; // fK x fK x fC_in x fC_out 크기를 가지는 4차원 배열
    double* bias_tensor; // fC_out 크기를 가지는 1차원 배열 (bias는 각 filter당 1개 존재)
public:
    Layer_Conv(string _name, int _fK, int _fC_in, int _fC_out, int init_type, string _filename_weight = "", string _filename_bias = "") { ... }
    void init(int init_type) { ... }
    ~Layer_Conv() override { ... }
    Tensor3D* forward(const Tensor3D* input) override { ... }
    void get_info(string& _name, int& _fK, int& _fC_in, int& _fC_out) const override { ... }
    void print() const override { ... }
};
```

Layer_Conv Class



기말프로젝트: 객체지향 CNN 구현

- Layer 객체: _2_main_layer.cpp를 구현

```
int main() {
    Layer* layer1 = new Layer_Conv("Conv1", 3, 3, 1, MEAN_INIT);
    Layer* layer2 = new Layer_ReLU("Relu1", 1, 1, 1);

    cout << "(Layer information)_____" << endl;
    layer1->print();
    layer2->print();
    cout << endl << endl;

    // tensor1 -> (layer1) -> tensor2 -> (layer2) -> tensor3
    Tensor3D* tensor1 = read_tensor("tensor_5x5x2.txt");
    Tensor3D* tensor2 = layer1->forward(tensor1);
    Tensor3D* tensor3 = layer2->forward(tensor2);
    cout << endl << endl;

    cout << "(Tensor information)_____" << endl;
    cout << "[tensor1]:" << endl;
    tensor1->print();
    print_all_elements((*tensor1));
    cout << endl;

    cout << "[tensor2]:" << endl;
    tensor2->print();
    print_all_elements((*tensor2));
    cout << endl;

    cout << "[tensor3]:" << endl;
    tensor3->print();
    print_all_elements((*tensor3));
    cout << endl;

    system("PAUSE");
    return 0;
}
```

_2_main_layer.cpp

```
(Layer information)_____  
Conv1:  3*3*3*1  
Relu1:  1*1*1*1  
  
Conv1 is finished  
Relu1 is finished
```

```
(Tensor information)_____  
[tensor1]:  
5*5*2  
0-th channel:  
  0.694  0.609 -0.151 -0.252 -0.0679  
  0.505  0.538  0.64  0.506  0.64  
  0.267  0.0882 -0.457  0.197  0.255  
  0.26 -0.654 -0.505 -0.671 -0.333  
 -0.0268 -0.148  0.301 -0.00506  0.0339  
1-th channel:  
  0.307  0.615 -0.336  0.215  0.2  
  0.25 -0.097 -0.403  0.279 -0.298  
 -0.502 -0.232 -0.0643 -0.657  0.158  
  0.23  0.299 -0.63 -0.651 -0.412  
  0.579 -0.61 -0.309 -0.245 -0.11  
  
[tensor2]:  
5*5*1  
0-th channel:  
  0 0 0 0 0  
  0 0.0841 0.0385 0.015 0  
  0 -0.0173 -0.0917 -0.0891 0  
  0 -0.0783 -0.183 -0.152 0  
  0 0 0 0 0  
  
[tensor3]:  
5*5*1  
0-th channel:  
  0 0 0 0 0  
  0 0.0841 0.0385 0.015 0  
  0 0 0 0 0  
  0 0 0 0 0  
  0 0 0 0 0
```

실행화면



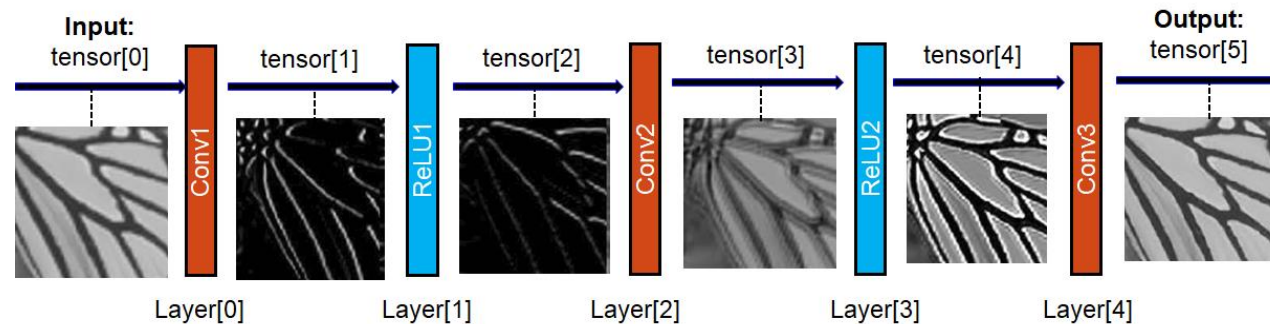
기말프로젝트: 객체지향 CNN 구현

- Model 객체: _3_main_model_mean.cpp를 구현
 - Layer와 Tensor를 vector로 관리 - 순차적으로 layer를 수행하고 출력 결과를 저장

```
class Model {
private:
    vector<Layer*> layers; //layer들을 순차적으로 저장
    vector<Tensor3D*> tensors; // tensor들을 순차적으로 저장 ( 0번째 tensor는 0번째 layer의 입력, 마찬가지로 1번째 tensor는 1번째 layer의 출력 )
public:
    Model() {}
    void add_layer(Layer* layer) { ... }
    ~Model() { ... }
    void test(string filename_input, string filename_output) { ... }

    void read_image(const string filename, byte*& pLoadImage, double**& img_Y, double**& img_U, double**& img_V, int& nH, int& nW)
    void save_image(string filename, byte*& pLoadImage, Tensor3D*& tensor_Y, double** img_U, double** img_V, int nH, int nW) { ... }
    void print_layer_info() const { ... }
    void print_tensor_info() const { ... }

    // void train();
};
```



기말프로젝트: 객체지향 CNN 구현

- Model 객체: _3_main_model_mean.cpp를 구현

```
int main() {  
    Model model;  
  
    // build model  
    model.add_layer(new Layer_Conv("Conv1", 3, 1, 1, MEAN_INIT));  
    model.add_layer(new Layer_ReLU("Relu1", 1, 1, 1));  
    model.add_layer(new Layer_Conv("Conv2", 3, 1, 1, MEAN_INIT));  
    model.add_layer(new Layer_ReLU("Relu2", 1, 1, 1));  
    model.add_layer(new Layer_Conv("Conv3", 3, 1, 1, MEAN_INIT));  
}
```

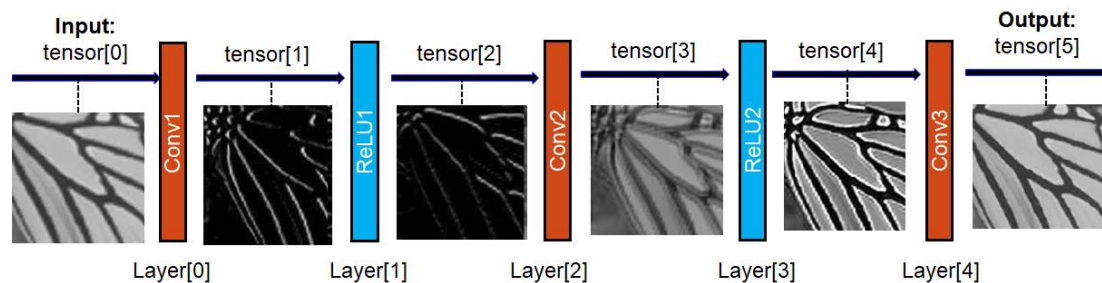
```
model.test("baby_512x512_input.bmp", "baby_512x512_output_mean.bmp");
```

```
model.print_layer_info();
```

```
model.print_tensor_info();
```

```
system("PAUSE");
```

```
return 0;
```



```
Reading (baby_512x512_input.bmp) is complete...
```

```
Conv1 is finished
```

```
Relu1 is finished
```

```
Conv2 is finished
```

```
Relu2 is finished
```

```
Conv3 is finished
```

```
Super-resolution is complete...
```

```
Saving (baby_512x512_output_mean.bmp) is complete...
```

```
(Layer information)
```

```
1-th layer: Conv1:      3*3*1*1
```

```
2-th layer: Relu1:      1*1*1*1
```

```
3-th layer: Conv2:      3*3*1*1
```

```
4-th layer: Relu2:      1*1*1*1
```

```
5-th layer: Conv3:      3*3*1*1
```

```
(Tensor information)
```

```
1-th tensor: 512*512*1
```

```
2-th tensor: 512*512*1
```

```
3-th tensor: 512*512*1
```

```
4-th tensor: 512*512*1
```

```
5-th tensor: 512*512*1
```

```
6-th tensor: 512*512*1
```

```
계속하려면 아무 키나 누르십시오 . . .
```

기말프로젝트: 객체지향 CNN 구현

- Model 객체: _3_main_model_mean.cpp를 구현
 - Test 결과: 더 blur된 영상이 생성(평균값 필터링을 적용)
 - 입력: baby_512x512_input.bmp
 - 출력: baby_512x512_output_mean.bmp(첨부된 파일과 동일하게 나오도록 구현)



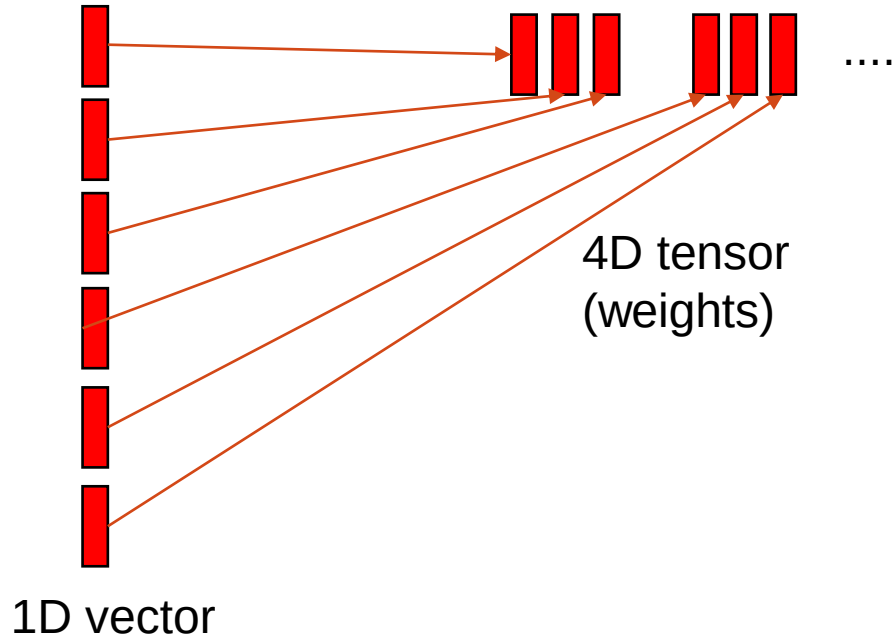
입력



출력

기말프로젝트: 객체지향 CNN 구현

- Model 객체: `_4_main_model_srcnn.cpp`를 구현
 - 최초 CNN 기반 SR 모델 (SRCNN: Super-Resolution CNN) [1]을 구현
 - 기존에 구현된 것에서 `Layer_Conv` 객체의 `init()` 멤버 함수의 `LOAD_INIT` 모드를 추가로 구현하면 완료됨
 - 파일(1D vector)로부터 읽은 값을 가중치(4D tensor)에 저장하는 순서에 유의 할 것
 - 행, 열, 채널, 필터 갯수 차원 순서로 채워짐



[1] Image Super-Resolution Using Deep Convolutional Networks (Dong et al., IEEE TPAMI, 2015)

기말프로젝트: 객체지향 CNN 구현

● Model 객체: _4_main_model_srcnn.cpp를 구현

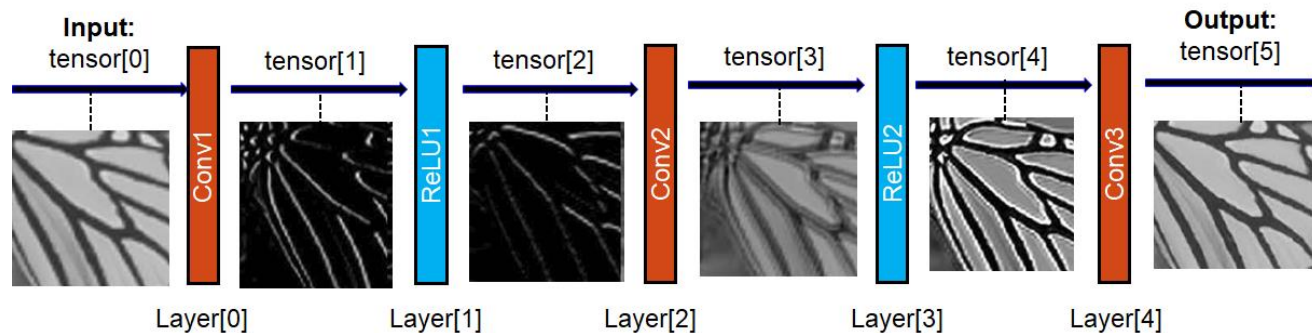
```
int main() {
    Model model;

    // build model
    model.add_layer(new Layer_Conv("Conv1", 9, 1, 64, LOAD_INIT, "model/weights_conv1_9x9x1x64.txt", "model/biases_conv1_64.txt"));
    model.add_layer(new Layer_ReLU("Relu1", 1, 64, 64));
    model.add_layer(new Layer_Conv("Conv2", 5, 64, 32, LOAD_INIT, "model/weights_conv2_5x5x64x32.txt", "model/biases_conv2_32.txt"));
    model.add_layer(new Layer_ReLU("Relu2", 1, 32, 32));
    model.add_layer(new Layer_Conv("Conv3", 5, 32, 1, LOAD_INIT, "model/weights_conv3_5x5x32x1.txt", "model/biases_conv3_1.txt"));

    model.test("baby_512x512_input.bmp", "baby_512x512_output_srcnn.bmp");

    model.print_layer_info();
    model.print_tensor_info();
    system("PAUSE");

    return 0;
}
```



./model 폴더 내부
(가중치/바이어스 값을 저장한 파일들)

biases_conv1_64.txt	2017-11-15 오전 10:00	텍스트 문서	2KB
biases_conv2_32.txt	2017-11-15 오전 10:00	텍스트 문서	1KB
biases_conv3_1.txt	2017-11-15 오전 10:01	텍스트 문서	1KB
weights_conv1_9x9x1x64.txt	2017-11-15 오전 10:03	텍스트 문서	106KB
weights_conv2_5x5x64x32.txt	2017-11-15 오전 10:06	텍스트 문서	1,060KB
weights_conv3_5x5x32x1.txt	2017-11-15 오전 10:07	텍스트 문서	17KB

```
Reading (baby_512x512_input.bmp) is complete...
Conv1 is finished
Relu1 is finished
Conv2 is finished
Relu2 is finished
Conv3 is finished
Super-resolution is complete...
Saving (baby_512x512_output_srcnn.bmp) is complete...
```

```
(Layer information)
1-th layer: Conv1: 9*9*1*64
2-th layer: Relu1: 1*1*64*64
3-th layer: Conv2: 5*5*64*32
4-th layer: Relu2: 1*1*32*32
5-th layer: Conv3: 5*5*32*1
```

```
(Tensor information)
1-th tensor: 512*512*1
2-th tensor: 512*512*64
3-th tensor: 512*512*64
4-th tensor: 512*512*32
5-th tensor: 512*512*32
6-th tensor: 512*512*1
계속하려면 아무 키나 누르십시오 . . .
```

기말프로젝트: 객체지향 CNN 구현

- Model 객체: `_4_main_model_srcnn.cpp`를 구현
 - Test 결과: 더 선명한 영상이 생성(SRCNN의 효과)
 - 입력: `baby_512x512_input.bmp`
 - 출력: `baby_512x512_output_srcnn.bmp`(첨부된 파일과 동일하게 나오도록 구현)



입력



출력

기말프로젝트: 객체지향 CNN 구현

- Model 객체: `_5_main_model_srcnn.cpp`를 구현
 - 기존에 구현된 forward pass에 backpropagation을 추가
 - `Layer_ReLU::backward()` — ReLU 역전파 (forward에서 양수였던 위치만 gradient 통과)
 - `Layer_Conv::backward()` — Convolution 역전파 (dL/dX , dL/dW , dL/db 계산)
 - `Layer_Conv::update_weights()` — SGD 가중치 업데이트 ($W = W - lr \times dL/dW$)
 - `Model::train()` — 학습 루프 (forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update 반복)
 - OpenMP 적용:
 - forward와 동일하게 backward, update_weights에도 `#pragma omp parallel for` 적용
 - thread 수(thread 수=2) 변화에 따른 처리속도(1 epoch 기준)를 측정하고 Speedup을 보고서에 기록할 것
 - 주의사항:
 - `_4_main_` 먼저 실행하여 `baby_512x512_output_srcnn.bmp`를 생성한 후 진행할 것
 - `before_train.bmp`와 `after_train.bmp`를 비교하여 학습 효과를 확인할 것
 - epoch마다 loss가 감소하는지 확인할 것



기말프로젝트: 객체지향 CNN 구현

- Model 객체: _5_main_model_srcnn.cpp를 구현

```
int main() {
    Model model;

    // build model (SRCNN 구조, MEAN_INIT으로 시작 → backprop으로 SRCNN 출력에 가까워지도록 학습)
    model.add_layer(new Layer_Conv("Conv1", 9, 1, 64, MEAN_INIT));
    model.add_layer(new Layer_ReLU("Relu1", 1, 64, 64));
    model.add_layer(new Layer_Conv("Conv2", 5, 64, 32, MEAN_INIT));
    model.add_layer(new Layer_ReLU("Relu2", 1, 32, 32));
    model.add_layer(new Layer_Conv("Conv3", 5, 32, 1, MEAN_INIT));

    // (1) train 전 inference 결과 저장
    model.test("baby_512x512_input.bmp", "baby_512x512_output_before_train.bmp");

    // (2) backpropagation 학습 수행
    //   입력: baby_512x512_input.bmp      (저해상도)
    //   정답: baby_512x512_output_srcnn.bmp (_4_main 실행 후 생성된 SRCNN 출력 영상)
    //   학습률(lr)과 epoch 수는 적절히 조정할 것
    model.train("baby_512x512_input.bmp", "baby_512x512_output_srcnn.bmp", 0.0001, 100);

    // (3) train 후 inference 결과 저장
    model.test("baby_512x512_input.bmp", "baby_512x512_output_after_train.bmp");

    model.print_layer_info();
    model.print_tensor_info();
    system("PAUSE");

    return 0;
}
```

출력 결과

```
Reading (baby_512x512_input.bmp) is complete...
Conv1 is finished ... Conv3 is finished
Saving (baby_512x512_output_before_train.bmp) is complete...

--- Training Start ---
Epoch [ 1/100] Loss: 0.052341 Time: X.XXs
Epoch [ 2/100] Loss: 0.048721 Time: X.XXs
...
Epoch [100/100] Loss: 0.012045 Time: X.XXs
Training complete.

Conv1 is finished ... Conv3 is finished
Saving (baby_512x512_output_after_train.bmp) is complete...

(Layer information)_____
...
(Tensor information)_____
...
계속하려면 아무 키나 누르십시오 . . .
```


기말프로젝트: 객체지향 CNN 구현

- Model 객체: `_5_main_model_srcnn.cpp`를 구현
 - Test 결과
 - 입력: `baby_512x512_input.bmp`
 - 타겟: `baby_512x512_output_srcnn.bmp`
 - 출력: `baby_512x512_output_backprop.bmp`



입력



출력

기말프로젝트: 객체지향 CNN 구현

성능 측정 코드

```
// 성능 측정 예시
double t_start, t_end;

t_start = omp_get_wtime();
model.train("input.bmp", "target.bmp", 0.001, 1); // 1 epoch (forward+backward)
t_end = omp_get_wtime();

cout << "Serial 1 epoch: " << (t_end - t_start) << " sec" << endl;

// OpenMP 활성화 후 동일하게 측정
// → 보고서에 thread 수별 speedup 표 작성
```

측정 항목	1 thread	2 threads	4 threads	Speedup
Forward pass	X.XX sec	X.XX sec	X.XX sec	X.XX ×
Backward pass	X.XX sec	X.XX sec	X.XX sec	X.XX ×
Weight update	X.XX sec	X.XX sec	X.XX sec	X.XX ×
Total (1 epoch)	X.XX sec	X.XX sec	X.XX sec	X.XX ×

제출 방법 및 기간

- e-campus로 아래 소스코드 및 PDF파일을 압축해서 제출
 - 소스코드: CTensor.h, CLayer.h, CModel.h, Closs.h
 - PDF: 각 main함수별로 결과물 (실행화면 스크린샷 및 출력 bmp파일)을 포함할 것
 - 제출 포맷: 이름_학번.zip
 - 제출 마감: _(늦게 제출된 결과물은 받지않음)
- 유의사항
 - main함수의 number에 따라 순서대로 구현할 것(_1_main_tensor.cpp, _2_main_layer.cpp...)
 - // (구현할것) ← 이라는 표시가 있는 부분에 구현(추가 함수 만들지 말것)
 - 각 main함수마다 동작 확인 후 on/off 방식으로 점수 부여 함
 - *visual studio를 활용할 것 .cpp는 소스파일 .h는 헤더파일에 업로드하여 진행*
 - imagelib.h 는 영상 데이터 입/출력 및 1/2/3/4차원 동적배열 할당 함수를 제공(수정 불가)

